

Proyecto final – Fase 2: Modelado y simulación de un enlace 16-QAM con ruido AWGN e interferencia

Jarol Nicolas Molano López - 2230391
William Camilo Motta Chacón - 2234639
Juan Andres Rojas Rueda - 2231065

https://github.com/J202R/2026_2_ComII_A1_A1E

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Universidad Industrial de Santander

6 de octubre de 2026

Abstract—This report presents the simulation of the 16-QAM link proposed in Phase 1 for data center interconnection. The conceptual and AWGN flowgraphs were run in GNU Radio at 32 kDd and 8 samples per symbol, and the complex envelope, PSD and constellation were recorded for noise amplitudes between 0.10 and 0.50. A Python model of an equivalent chain, with root-raised-cosine (RRC) filters and a matched filter, was used to obtain BER vs SNR curves with AWGN and with an adjacent 16-QAM channel ($SIR = 0$ dB, $\Delta f = 36$ kHz). The simulated BER matched the theoretical Gray-coded curve (12.19 dB for $BER = 10^{-4}$); the laboratory symbol table required 0.57 dB more E_b/N_0 , and the adjacent channel added a penalty of 2.3 dB.

Index Terms—16-QAM, AWGN, BER, GNU Radio, RRC filter

I Introducción

El tráfico entre centros de datos, impulsado por la replicación de almacenamiento y las cargas de inteligencia artificial, exige enlaces de interconexión (DCI) de mayor capacidad sin multiplicar la fibra instalada. En la Fase 1 se propuso 16-QAM para este escenario, ya que transmite 4 bits por símbolo y duplica la eficiencia espectral de QPSK [1], [2]. A cambio, sus 16 símbolos quedan más próximos entre sí, por lo que el ruido acumulado en el enlace y la diafonía entre canales DWDM vecinos provocan errores con mayor facilidad.

El objetivo de esta fase fue cuantificar esa sensibilidad mediante simulación antes de pasar a los radios definidos por software. Para ello se ejecutaron en GNU Radio los flujogramas base de la guía [3] con la tabla 16-QAM asignada, se observó la envolvente compleja, la PSD y la constelación para cinco niveles de ruido AWGN y se relacionó cada nivel con una SNR equivalente. Como los flujogramas no incluyen receptor, la curva BER vs SNR y el efecto de la interferencia controlada se obtuvieron

con un modelo en Python de una cadena equivalente.

II Metodología

II-A Flujogramas en GNU Radio

Se usaron los dos flujogramas de la guía [3] con la tabla 16-QAM asignada y los parámetros de la Tabla 1. El flujograma conceptual (Fig. 1a) genera bits aleatorios, los empaqueta (*Pack K Bits*, $K = 8$), los agrupa de a 4 (*Packed to Unpacked*), los mapea con *Chunks to Symbols* y los interpola con un pulso rectangular de 8 muestras. El flujograma AWGN (Fig. 2a) suma a esa señal un *Noise Source* gaussiano de amplitud `noise_amp`, que se fijó en 0.10, 0.25, 0.32, 0.40 y 0.50; el valor 0.32 es el punto nominal (Sec. II-B).

Tabla 1: Parámetros de la simulación

Parámetro	Valor
Tasa de símbolos R_s	32 kDd
Muestras por símbolo S_{ps}	8
Frecuencia de muestreo $f_s = R_s S_{ps}$	256 kHz
Tasa de bits $R_b = 4R_s$	128 kb/s
Potencia media de la constelación E_s	10
<code>noise_amp</code>	0.10, 0.25, 0.32, 0.40, 0.50
Filtro RRC (Python)	$\beta = 0.35$
Separación del canal vecino Δf	36 kHz
Relación señal a interferencia	$SIR = 0$ dB
Rango de E_b/N_0 (Python)	0–16 dB

II-B Relación entre `noise_amp` y la SNR

El *Noise Source* complejo reparte la amplitud A entre I y Q [4], por lo que la potencia del ruido es A^2 y la SNR por muestra es E_s/A^2 . Un filtro acoplado promedia las S_{ps} muestras de cada símbolo, con $N_0 = A^2/S_{ps}$, de modo que

$$\frac{E_s}{N_0} = S_{ps} \frac{E_s}{A^2}, \quad \frac{E_b}{N_0} = \frac{1}{4} \frac{E_s}{N_0}. \quad (1)$$

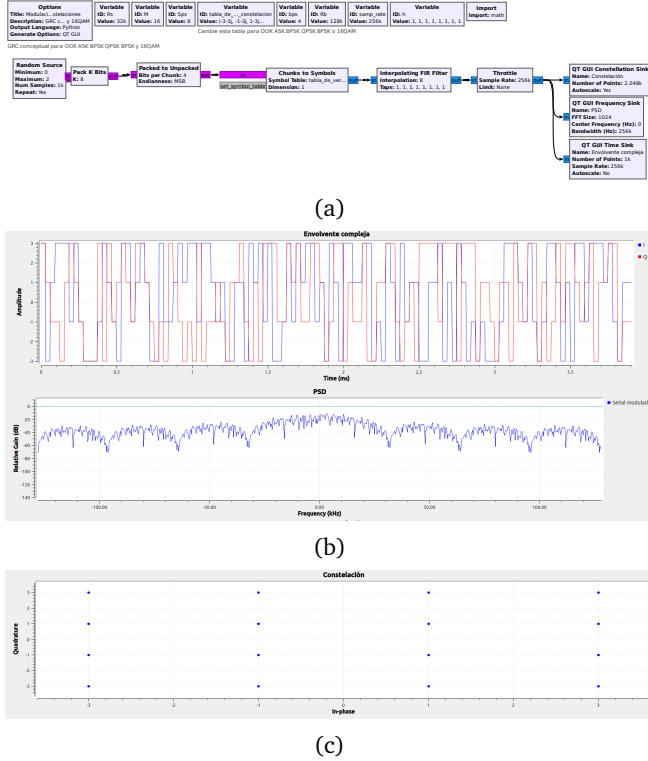


Figura 1: Simulación sin ruido en GNU Radio: (a) flujograma conceptual; (b) envolvente compleja y PSD; (c) constelación.

La BER teórica de 16-QAM con codificación Gray es [5], [6]

$$P_b \approx \frac{3}{8} \operatorname{erfc} \left(\sqrt{\frac{2 E_b}{5 N_0}} \right). \quad (2)$$

En GNU Radio el pulso rectangular de S_{ps} muestras tiene energía S_{ps} , por lo que un filtro acoplado añadiría $10 \log_{10} S_{ps} = 9$ dB; sin él, la decisión sobre una muestra corresponde a $E_b/N_0 = (E_s/A^2)_{\text{dB}} - 6$ dB. En Python el RRC se normaliza a energía unitaria, de modo que $E_s/N_0 = E_s/\sigma^2$ a la salida del filtro acoplado.

La selección de $\text{noise_amp} = 0.32$ busca emular, mediante un modelo AWGN, el efecto neto del ruido óptico acumulado, la distancia del enlace y la reducción del OSNR, propios de los enlaces sobre fibra óptica usados en interconexión entre centros de datos. En estos se requiere un E_b/N_0 con margen sobre el umbral teórico de 12.19 dB ($\text{BER} = 10^{-4}$) para obtener una tasa de error prácticamente nula tras el filtrado acoplado. Se fijó $\text{noise_amp} = 0.32$, que equivale a $E_b/N_0 \approx 22.9$ dB con filtro acoplado ($\gamma_b = 195.88$), es decir, 10.73 dB sobre el umbral. Despejando A de la Ec. (1) con ese γ_b ,

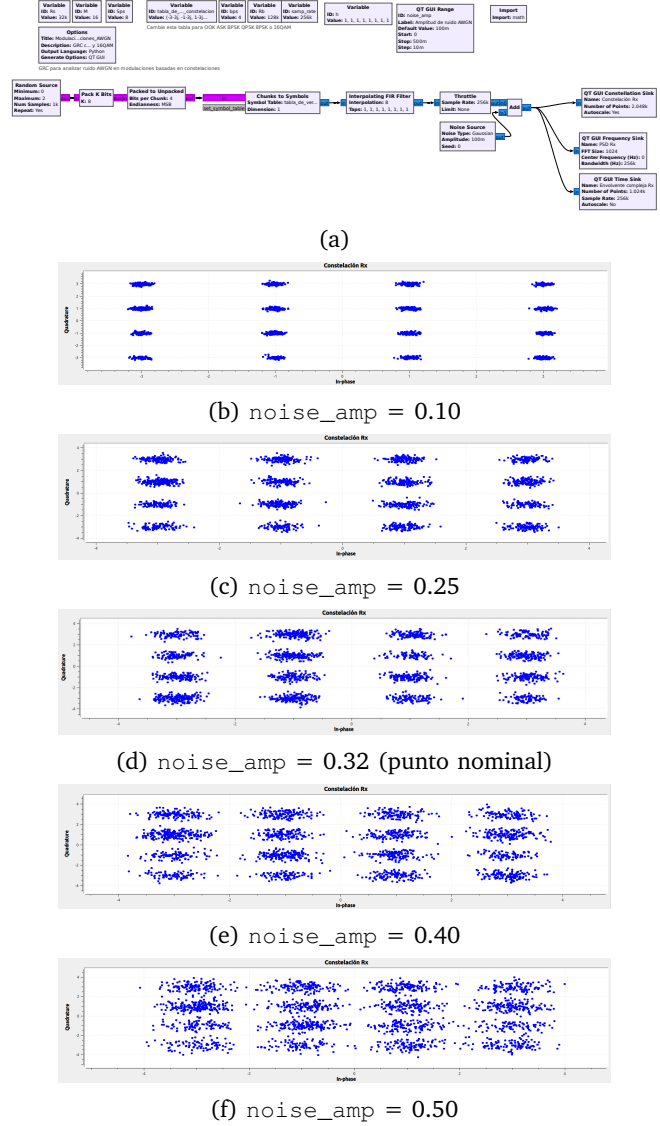


Figura 2: Simulación con AWGN en GNU Radio: (a) flujograma; (b)–(f) constelación recibida (todas las muestras, sin filtro acoplado).

$E_s = 10$ y $S_{ps} = 8$:

$$A = \sqrt{\frac{S_{ps} E_s}{4 \gamma_b}} = \sqrt{\frac{8 \cdot 10}{4 \cdot 195.88}} \approx 0.3195 \approx 0.32. \quad (3)$$

Así, $\text{noise_amp} = 0.32$ equivale a una $\text{SNR}_{\text{muestra}} = E_s/A^2 = 19.9$ dB. Con filtro acoplado esto da $E_b/N_0 \approx 22.9$ dB; sin él, como en la constelación de GNU Radio (Fig. 2d), corresponde a $E_b/N_0 \approx 13.9$ dB, y aun así las muestras prácticamente no cruzan los umbrales de decisión (0 y ± 2): la BER es de 8.1×10^{-6} con una muestra por símbolo (Tabla 2).

En el modelo de Python el RRC se normaliza a energía unitaria, por lo que $E_b/N_0 = 14$ dB ($E_s/N_0 = 20$ dB) implica $\sigma^2 = E_s/100 = 0.1$, es decir, $A \approx 0.32$: las Figs. 8 y 9 corresponden al mismo ruido que la Fig. 2d. La diferencia entre 13.9 dB y 14 dB es solo el redondeo de A .

II-C Modelo en Python para la curva BER

Como los flujogramas no tienen receptor, la BER se calculó en Python [7], [8] con la misma tabla y parámetros, agregando filtro RRC en Tx, filtro acoplado RRC en Rx, muestreo en el instante óptimo, detección por mínima distancia y conteo de bits erróneos (Listado 1). Se simuló 1.2×10^6 a 3.6×10^6 bits por punto. Se comparó la tabla del laboratorio con una codificación Gray (Fig. 3) y se agregó una interferencia definida por $SIR = P_s/P_i$: un canal 16-QAM vecino de igual potencia separado 36 kHz, que emula la diafonía DWDM.

Listing 1: Cadena Tx–canal–Rx en Python

```
1 idx = bits_a_indices(bits)
2 x = tx_rrc(tabla[idx]) # Tx RRC
3 r = x + awgn(len(x), ES/esn0, rng) # canal
4 r += interferencia # canal vecino
5 y = rx_rrc(r, n_sim) # filtro acoplado
6 ber = np.mean(indices_a_bits(detector(y))
    != bits)
```

II-D Codificación Gray frente a la tabla del laboratorio

Se optó por comparar la tabla 16-QAM asignada con su equivalente en codificación Gray. El flujograma de GNU Radio asigna cada grupo de 4 bits al punto que ocupa esa posición en la tabla `tabla_de_verdad_constelacion` y no incluye ningún bloque que aplique la codificación Gray. La tabla de la guía tampoco la cumple (Fig. 3a): dos símbolos vecinos difieren en promedio en 2.33 bits y, en el peor caso, en 4. Por ello, un error de símbolo hacia un punto adyacente puede producir varios bits erróneos y la BER aumenta. La codificación Gray (Fig. 3b) se obtiene reordenando la tabla: los dos primeros bits seleccionan el nivel de I y los dos últimos el de Q, con el orden 00, 01, 11, 10 para $-3, -1, 1, 3$, de modo que los puntos adyacentes difieren en un solo bit.

II-E Interferencia de canal adyacente

Adicionalmente se introdujo una fuente de interferencia controlada, acorde con el ámbito de la aplicación. En las fibras ópticas empleadas para la interconexión de

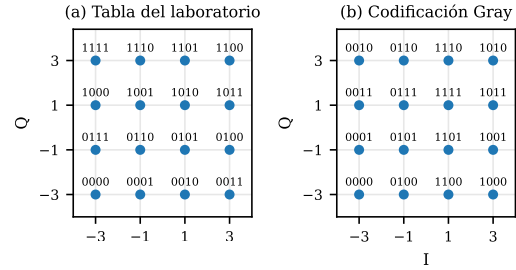


Figura 3: Asignación de bits a los símbolos 16-QAM: (a) tabla del laboratorio; (b) codificación Gray (los dos primeros bits seleccionan I y los dos últimos Q).

centros de datos se multiplexan varias transmisiones en longitud de onda, y aunque cada canal ocupa su propia longitud de onda, el crosstalk en los multiplexores o los efectos no lineales de la fibra pueden hacer que parte de la energía de un canal vecino llegue al receptor. Esta situación equivale a una multiplexación deficiente y para representarla se adicionó a la señal recibida un segundo canal 16-QAM, con el mismo filtro RRC y la misma potencia, desplazado en frecuencia, y la interferencia se cuantificó mediante la relación señal a interferencia, $SIR = P_s/P_i$ [9], evaluando su efecto sobre la BER y sobre la constelación a la salida del filtro acoplado con $\Delta f = 36$ kHz e igual potencia ($SIR = 0$ dB).

III Resultados y análisis

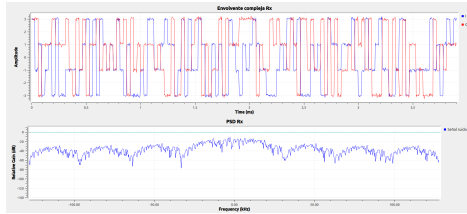
III-A Señal sin ruido

En la Fig. 1b, I y Q toman solo los niveles ± 1 y ± 3 y la PSD tiene forma sinc^2 con nulos cada $R_s = 32$ kHz. La constelación de la Fig. 1c muestra los 16 puntos de la tabla.

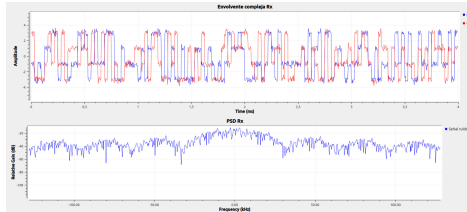
III-B Efecto del ruido AWGN

Las Figs. 2b–2f muestran que cada símbolo se convierte en una nube: cerca de ± 0.2 con $A = 0.10$ y hasta ± 1 con $A = 0.50$, donde algunas muestras cruzan los umbrales en 0 y ± 2 . En la Fig. 4, I y Q fluctúan cada vez más y el ruido llena los nulos de la PSD; según la Ec. (1), el piso queda 39.0 dB bajo el pico con $A = 0.10$ y 25.1 dB con $A = 0.50$ (Fig. 4c).

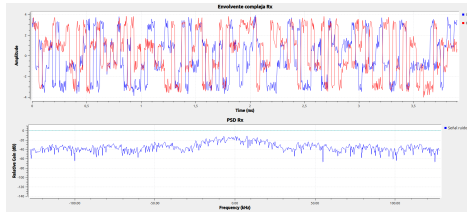
En la Tabla 2, las columnas de SNR se calcularon con la Ec. (1); por ejemplo, con $A = 0.50$ la SNR por muestra es $10/0.5^2 = 40$ (16.0 dB) y, al sumar $10 \log_{10} 8 = 9.0$ dB del filtro acoplado y restar $10 \log_{10} 4 = 6.0$ dB, se obtiene $E_b/N_0 = 19.0$ dB. Las columnas de BER se obtuvieron en Python reproduciendo la cadena del flujograma AWGN (pulso rectangular, tabla del laboratorio



(a) noise_amp = 0.10



(b) noise_amp = 0.32



(c) noise_amp = 0.50

Figura 4: Envelope compleja y PSD con AWGN en GNU Radio.

y ruido de amplitud A) con 1.6×10^6 bits por caso, decidiendo con una sola muestra por símbolo, como la constelación de GNU Radio, o con filtro acoplado. Sin filtro acoplado la BER llega a 4.0×10^{-3} con $A = 0.50$, mientras que con filtro acoplado no hubo errores en ningún caso. Para llegar a $E_b/N_0 = 10$ dB se necesitaría $A \approx 1.41$, fuera del rango del deslizador (0 a 0.5).

Tabla 2: SNR calculada con la Ec. (1) y BER simulada en Python para cada noise_amp

A	SNR/muestra [dB]	E_b/N_0 1 muestra [dB]	E_b/N_0 con FA [dB]	BER 1 muestra	BER con FA
0.10	30.0	24.0	33.0	$< 6.3 \times 10^{-7}$	$< 6.3 \times 10^{-7}$
0.25	22.0	16.0	25.1	$< 6.3 \times 10^{-7}$	$< 6.3 \times 10^{-7}$
0.32	19.9	13.9	22.9	8.1×10^{-6}	$< 6.3 \times 10^{-7}$
0.40	18.0	12.0	21.0	3.3×10^{-4}	$< 6.3 \times 10^{-7}$
0.50	16.0	10.0	19.0	4.0×10^{-3}	$< 6.3 \times 10^{-7}$

FA: filtro acoplado. BER con la tabla del laboratorio. $< 6.3 \times 10^{-7}$: cero errores en 1.6×10^6 bits.

III-C Curva BER vs SNR

En la Fig. 5, la curva simulada con codificación Gray coincide con la Ec. (2) (12.19 dB para $BER = 10^{-4}$). La

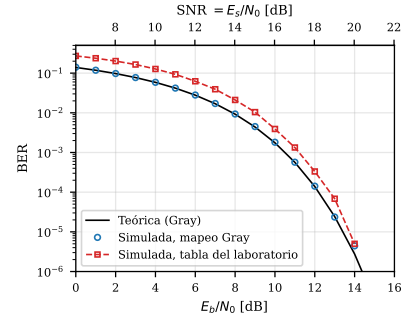


Figura 5: BER vs SNR en AWGN: teórica (Gray) y simuladas con codificación Gray y con la tabla del laboratorio.

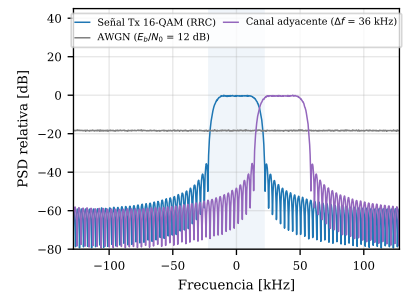


Figura 6: PSD de la señal 16-QAM con filtro RRC, el ruido AWGN y el canal vecino. La franja sombreada es el ancho de banda $B_s(1 + \beta)$.

tabla del laboratorio no es Gray: sus vecinos difieren hasta en 4 bits (Fig. 3a), por lo que multiplica por aproximadamente 2 la BER a 0 dB (0.27 frente a 0.14) y requiere 0.57 dB más de E_b/N_0 (Tabla 3).

III-D Efecto de la interferencia

La Fig. 6 muestra que el filtro RRC confina la señal en ± 21.6 kHz y que el canal vecino se solapa con su borde. En la Fig. 7, el canal vecino añade 2.3 dB (Tabla 3). Esto se ve en las nubes ensanchadas de la Fig. 9c.

III-E Diagrama de ojo y constelaciones

Los diagramas de ojo de la Fig. 8 muestran que el par RRC en Tx y filtro acoplado en Rx no introduce interferencia entre símbolos: sin ruido, los cuatro niveles por eje se distinguen en el instante de muestreo. Con $E_b/N_0 = 14$ dB los ojos se estrechan, pero siguen abiertos, lo que es consistente con una BER del orden de 10^{-6} (Fig. 5). En la Fig. 9b las nubes quedan separadas, mientras que con el canal vecino (Fig. 9c) se ensanchan y algunos puntos se acercan a los umbrales de decisión.

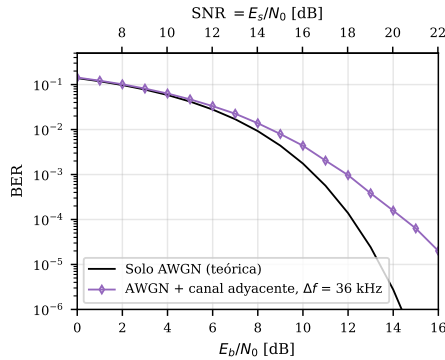


Figura 7: BER vs SNR (Gray) con canal 16-QAM vecino.

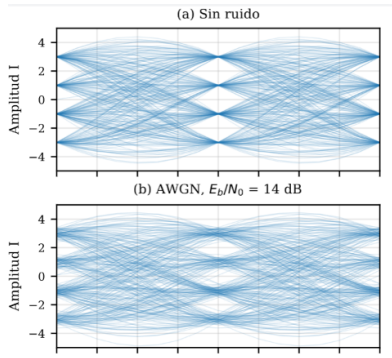


Figura 8: Diagrama de ojo (componente I) a la salida del filtro acoplado: (a) sin ruido; (b) AWGN, $E_b/N_0 = 14$ dB.

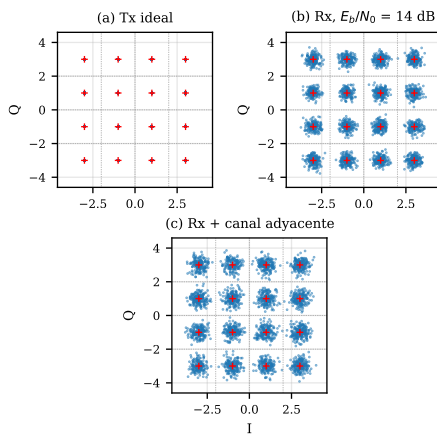


Figura 9: Constelaciones a la salida del filtro acoplado: (a) Tx ideal (equivalente al Rx sin ruido, por el criterio de Nyquist); (b) AWGN, $E_b/N_0 = 14$ dB ($E_s/N_0 = 20$ dB); (c) AWGN y canal vecino, $\Delta f = 36$ kHz, SIR = 0 dB.

IV Conclusiones

Con $A = 0.50$ la constelación de GNU Radio (Fig. 2f) alcanza $E_b/N_0 = 10$ dB y una BER de 4.0×10^{-3}

Tabla 3: E_b/N_0 requerido para BER = 10^{-4}

Escenario	E_b/N_0 [dB]	Penalización [dB]
Teórico (Gray)	12.19	–
AWGN, Gray	12.19	0.00
AWGN, tabla del laboratorio	12.76	0.57
AWGN + canal vecino	14.50	2.31

al decidir sobre una muestra por símbolo; con filtro acoplado el mismo ruido equivale a $E_b/N_0 = 19$ dB y no produjo errores (Tabla 2), por lo que, con receptor completo, el rango de 0 a 0.5 no alcanza la zona útil de la curva BER. Además, el par RRC–filtro acoplado no introdujo interferencia entre símbolos (Fig. 8).

La curva simulada con codificación Gray coincidió con la teórica (Fig. 5); la tabla del laboratorio requirió 0.57 dB adicionales de E_b/N_0 , por lo que conviene usar codificación Gray.

El canal vecino separado 36 kHz añadió una penalización de 2.3 dB (Tabla 3); como su espectro se solapa con la banda de la señal, se espera que la separación deba superar $R_s(1 + \beta) = 43.2$ kHz, valor a partir del cual los espectros dejan de solaparse; este umbral no se verificó con un barrido de Δf . En la Fase 3 se espera verificar con los radios definidos por software la penalización del orden de 2.3 dB y la mejora de la codificación Gray.

Referencias

- [1] W. C. M. Chacón, J. N. M. López, and J. A. R. Rueda, “Fase 1: Análisis de la modulación 16-QAM aplicada a enlaces ópticos para interconexión de centros de datos,” Proyecto final de Comunicaciones II, Universidad Industrial de Santander, 2026.
- [2] P. J. Winzer and R.-J. Essiambre, “Advanced optical modulation formats,” *Proceedings of the IEEE*, vol. 94, no. 5, pp. 952–985, 2006.
- [3] O. Tijaro and R. Zamora, “Proyecto final de Comunicaciones II – Fase 2: Modelado y simulación (Guía 1). Flujogramas base para modulaciones digitales en GNU Radio,” Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones, Universidad Industrial de Santander, 2026.
- [4] GNU Radio Project, “Noise source,” https://wiki.gnuradio.org/index.php/Noise_Source, consultado: oct. 2026.
- [5] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. New York: McGraw-Hill, 2008.
- [6] B. Sklar, *Digital Communications: Fundamentals and Applications*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2001.
- [7] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [8] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [9] A. Goldsmith, *Wireless Communications*. Cambridge, U.K.: Cambridge University Press, 2005.
- [10] K. Kikuchi, “Fundamentals of coherent optical fiber communications,” *Journal of Lightwave Technology*, vol. 34, no. 1, pp. 157–179, 2016.